CyberShield: A 2-Tier Cascade Network Intrusion Detection System

Muhammad Tayyab Nasir, Tehreem Mazhar
BSSE23018-A, BSSE23086-A
Supervised by Dr, Ahmed Raza
Department of Computer Science
Information Technology University (ITU), Punjab, Pakistan

***Abstract*—This paper presents CyberShield, a 2-Tier Cascade Network Intrusion Detection System (NIDS) engineered to overcome class imbalance and computational constraints inherent in real-world network security deployments. The system is trained on the CICIDS2017 benchmark after a rigorous data-engineering pipeline that eliminated a critical data-leakage feature, 307,078 duplicate records, and Unicode label corruption, producing a mathematically pure corpus of 2,520,798 network flow records. Tier B deploys a balanced Random Forest classifier as a generalized multi-class threat hunter; Tier C deploys a sample-weighted XGBoost deep scanner as the ultimate safety net for ultra-rare attack categories. Tier B achieves a Macro F1 score of 0.8605 (99.83% accuracy); Tier C achieves 0.8821 (99.86% accuracy) with perfect 1.00 recall on both Heartbleed and Infiltration attacks. The architecture is validated in a hybrid edge/cloud deployment: a native C++ ONNX Runtime engine on Android delivers sub-2 ms inference, while a Python FastAPI microservice on AWS Elastic Beanstalk serves web clients.**

***Index Terms*—Network Intrusion Detection, CICIDS2017, Random Forest, XGBoost, Class Imbalance, Cascade Architecture, ONNX Runtime, Edge Deployment, Anomaly Detection**

I. Introduction

Modern enterprise networks generate millions of packets per second, creating an asymmetric challenge for security systems. Traditional signature-based Intrusion Detection Systems (IDS) fail against novel attack patterns, motivating the adoption of machine-learning approaches that generalize beyond known signatures [1]. The CICIDS2017 dataset [2], produced by the Canadian Institute for Cybersecurity, is the *de facto* benchmark for modern NIDS research. It captures 2.8 million labeled network flows spanning 14 attack categories including DDoS, PortScan, Brute-Force, Botnet, Infiltration, and the rare Heartbleed exploit.

The extreme class imbalance in CICIDS2017—BENIGN traffic exceeds 80% of all records while Heartbleed contains only 11 examples—makes naive accuracy a misleading evaluation metric and exposes standard deep-learning pipelines to majority-class collapse. This work presents CyberShield, a 2-Tier Gradient-Boosted Cascade that addresses these challenges through rigorous data engineering and class-balancing strategies embedded directly into the model training objectives.

The main contributions of this paper are:

- A reproducible data-engineering pipeline (`clean_data_final.py`) that eliminates leakage, duplicates, and Unicode corruption from CICIDS2017.
- A 2-Tier Cascade combining a balanced Random Forest (Tier B) and a sample-weighted XGBoost deep scanner (Tier C).
- A hybrid edge/cloud deployment: native C++ ONNX Runtime on Android (sub-2 ms) and a Python FastAPI microservice on AWS Elastic Beanstalk.
- Empirical evidence that tree-based ensembles with class-balancing strategies decisively outperform deep-learning baselines on imbalanced tabular data.

II. Related Work

A. Machine Learning for Intrusion Detection

Tavallaee et al. [3] demonstrated that Random Forests consistently outperform neural networks on KDD Cup 1999 due to majority-class collapse under severe imbalance. Sharafaldin et al. [2] confirmed this on CICIDS2017, achieving strong F1 scores on common attacks but near-zero recall on rare categories such as Heartbleed and Infiltration. These findings directly motivate the cascade architecture proposed in this work.

B. Class Imbalance Mitigation

Resampling methods such as SMOTE [4] address imbalance at the data level but introduce synthetic artifacts in high-dimensional network feature spaces. Cost-sensitive learning—exemplified by XGBoost's `scale_pos_weight` and SKLEARN's `compute_sample_weight` [5]—embeds imbalance correction directly into the loss function, providing a more principled solution for tabular network data. CyberShield adopts this latter strategy exclusively.

C. Edge Deployment of ML Models

The Open Neural Network Exchange (ONNX) format [6] enables hardware-agnostic model deployment. Kang et al. [7] demonstrated that ONNX Runtime in C++ achieves 10–100$\times$ lower inference latency versus Python wrappers on ARM-based mobile processors, validating the architectural decision to build a native Android engine rather than relying on an on-device Python interpreter.

III. Dataset & Data Engineering

A. CICIDS2017 Overview

CICIDS2017 [2] contains 2,830,743 raw network flow records with 78 features extracted by CICFlowMeter, capturing packet-level statistics including flow duration, byte counts, inter-arrival times, flag ratios, and subflow metrics across 14 labeled attack categories.

B. Diagnostic Audit — Three Critical Flaws

A pre-training audit revealed three critical flaws that would produce misleading evaluation metrics if unaddressed.

Data Leakage. An engineered binary feature `Is_Attack` was present in the training matrix, encoding the ground-truth label directly. Any classifier achieved $\geq$99.9% accuracy by memorizing this lookup rather than learning network behavior.

Duplicate Records. Exactly 307,078 duplicate rows artificially inflate validation scores by leaking training examples into the evaluation split under random partitioning.

Label Mapping Inconsistency. Misaligned integer mappings between independently trained models caused catastrophic ensemble failures—DDoS mapping to class 1 in one model and class 2 in another, producing nonsensical cascade outputs.

C. Remediation Pipeline

The `clean_data_final.py` pipeline implements four steps: (1) a hardcoded JSON label map as the singular source of truth for integer-to-class mappings; (2) regular-expression Unicode sanitization stripping invisible zero-width spaces and non-breaking hyphens; (3) exact-row deduplication via `DataFrame.drop_duplicates`; and (4) explicit removal of the `Is_Attack` column before any feature matrix is constructed. The resulting dataset contains 2,520,798 rows—a 10.96% reduction—with verified label integrity.

TABLE I
CICIDS2017 Post-Clean Dataset Composition

Attack Category	Clean Count	% Total
BENIGN	2,018,340	80.08%
DoS Hulk	205,139	8.14%
PortScan	141,259	5.60%
DDoS	113,705	4.51%
DoS GoldenEye	9,145	0.36%
FTP-Patator	7,048	0.28%
SSH-Patator	5,238	0.21%
DoS Slowloris	5,148	0.20%
DoS Slowhttptest	4,885	0.19%
Bot	1,746	0.07%
Web Attack – BF	1,339	0.05%
Web Attack – XSS	579	0.02%
Infiltration	32	<0.01%
Heartbleed	11	<0.01%
TOTAL	**2,520,798**	**100%**

IV. System Architecture

CyberShield implements a 2-Tier Cascade architecture. No single classifier can simultaneously optimize precision, recall, and computational efficiency across all 14 attack categories. The cascade separates threat hunting (Tier B) from deep anomaly scanning (Tier C), allowing independent optimization of each tier for its specific objective.

A. Tier B — The Enforcer (Random Forest)

Tier B deploys a Random Forest classifier [9] across all 78 network features. Its role is generalized multi-class threat classification with high precision on common attack vectors. The `class_weight='balanced'` parameter automatically computes per-class weights inversely proportional to class frequency:

$$w_c = \frac{n_{\text{samples}}}{n_{\text{classes}} \times n_c}, \tag{1}$$

preventing majority-class dominance without oversampling.

TABLE II
Tier B — Random Forest Hyperparameters

Parameter	Value
`n_estimators`	75
`max_depth`	25
`class_weight`	`'balanced'`
`n_jobs`	-1 (parallel)
`random_state`	42

B. Tier C — The Deep Scanner (XGBoost)

Tier C deploys an XGBoost classifier [8] with dynamic sample weighting via `compute_sample_weight('balanced', y_train)`. This per-sample weight vector is passed to XGBoost's `fit()` method, forcing the gradient boosting algorithm to heavily penalize misclassifications of ultra-rare attack classes. The `hist` tree method reduces training complexity from $\mathcal{O}(n \times d)$ to $\mathcal{O}(b \times d)$ where $b = 256$ histogram bins, enabling efficient training on 2.5 million rows.

TABLE III
Tier C — XGBoost Deep Scanner Hyperparameters

Parameter	Value
`n_estimators`	200
`max_depth`	8
`learning_rate`	0.1
`tree_method`	`'hist'`
`sample_weights`	`compute_sample_weight`
`eval_metric`	`'mlogloss'`

C. Cascade Decision Logic

Both tiers are trained independently on the same clean dataset. At inference time: (1) Tier B produces a multi-class prediction with a confidence score. (2) If confidence falls below threshold $\theta = 0.70$, the flow is escalated to Tier C. (3) Tier C's prediction is reported for escalated flows; Tier B's for high-confidence cases. This ensures Tier C's superior sensitivity to rare attacks is applied precisely where Tier B exhibits uncertainty.

V. Deployment Architecture

A. ONNX Model Export

Both SKLEARN/XGBoost models are exported to ONNX format using `skl2onnx` and `onnxmltools` respectively. ONNX decouples inference from the Python training environment, enabling deployment to arbitrary hardware without framework dependencies.

B. Edge Deployment — Android C++ Engine

A native C++ inference engine (`native_bridge.cpp`) is compiled as a shared library targeting Android ABI `arm64-v8a`. It loads ONNX models directly into mobile RAM via `onnxruntime_cxx_api.h` with `Ort::MemoryInfo` for zero-copy tensor allocation. Flutter communicates with the engine through Dart Foreign Function Interface (FFI) memory pointers, achieving fully offline, air-gap-capable inference.

C. Cloud Deployment — AWS FastAPI Microservice

A Python FastAPI application (`api_server.py`) exposes a REST endpoint (`/classify`) accepting a JSON array of 78 network features. It preprocesses inputs through the same scaling pipeline used during training and returns a predicted attack category with confidence scores. The service is deployed to AWS Elastic Beanstalk, providing auto-scaling capacity for web browser clients that cannot execute native C++ binaries.

TABLE IV
EDGE VS. CLOUD DEPLOYMENT COMPARISON

Feature	C++ Edge (Android)	Python Cloud (AWS)
Inference Engine	`onnxruntime_cxx_api`	`onnxruntime` (Python)
Preprocessing	`std::vector`	`pandas` / `numpy`
Latency/Packet	< 2 ms	50–200 ms
Memory	Ultra-light	Heavy (interpreter)
Connectivity	Offline / Air-gap	Internet required
Target Client	Android App	Web Browser

VI. Experimental Results

A. Evaluation Methodology

The clean 2,520,798-row dataset was partitioned using a stratified 80/20 train/test split, preserving class distribution across all 14 categories. Four metrics are reported: overall accuracy, Macro-averaged F1 score (unweighted mean across all 14 classes), and per-class recall for the two most critical rare attacks—Heartbleed and Infiltration.

Macro F1 is the primary headline metric because it weights each class equally regardless of support size, penalizing any model that ignores rare attacks. A score of 1.00 requires perfect precision and recall on every attack category, including Heartbleed with its 11-sample total support.

B. Baseline: PyTorch MLP (Pre-Engineering)

Prior to data-engineering remediation, a three-layer MLP trained on the contaminated dataset achieved 99.99% accuracy—attributable entirely to the `Is_Attack` data leak—but a catastrophic Macro F1 of 0.0669. Post-remediation evaluation on the clean dataset revealed complete majority-class collapse: the network predicted BENIGN for virtually all inputs, yielding 0.00 recall on both Heartbleed and Infiltration.

C. Tier B and Tier C Results

Tier B's Random Forest achieves 99.83% accuracy with Macro F1 $= 0.8605$. Infiltration recall reaches 1.00 and Heartbleed recall reaches 0.50. Tier C's XGBoost achieves 99.86% accuracy with Macro F1 $= 0.8821$—a 2.5% improvement—and perfect 1.00 recall on both rare attack categories. The `compute_sample_weight` strategy treats a single Heartbleed misclassification as approximately 230,000 times more costly than a BENIGN miss.

TABLE V
COMPREHENSIVE MODEL PERFORMANCE COMPARISON

Metric	MLP (Leaky)	MLP (Clean)	Tier B RF	Tier C XGB
Accuracy	99.99%*	82.41%	99.83%	99.86%
Macro F1	0.0669 ×	0.1134 ×	0.8605 ✓	0.8821 ✓
Heartbleed Recall	0.00 ×	0.00 ×	0.50	1.00 ✓
Infiltration Recall	0.00 ×	0.00 ×	1.00 ✓	1.00 ✓
Data Integrity	Leaky	Pure	Pure	Pure

*Accuracy inflated by `Is_Attack` data leak — not a valid measure of generalisation performance.

VII. Discussion

A. Why Tree Ensembles Outperform MLPs

Standard gradient descent minimizes cross-entropy loss across the full dataset. With 80% BENIGN samples, predicting BENIGN for every input achieves 80% accuracy and a loss converging toward $\log(0.8) \approx 0.22$—a local optimum gradient descent readily finds. Tree-based splitting criteria (Gini impurity, information gain) evaluate each feature threshold independently, and class-balancing mechanisms directly reweight the splitting criterion rather than an aggregate gradient.

B. Cascade Architecture Advantages

The 2-tier cascade provides three operational advantages over a monolithic classifier: (1) *Independent calibration*—Tier B can be retrained without invalidating Tier C's rare-attack specialization. (2) *Computational efficiency*—high-confidence benign traffic is resolved at Tier B without incurring full XGBoost evaluation cost. (3) *Interpretability*—Random Forest feature importance rankings at Tier B provide actionable intelligence for network administrators.

C. Limitations and Future Work

The primary limitation is temporal generalization: CICIDS2017 traffic was captured in 2017 and may not represent modern attack vectors such as encrypted command-and-control channels. The 11-sample Heartbleed support requires cautious

interpretation of the perfect recall result. Future work will pursue: (1) adaptive threshold calibration via Platt scaling; (2) federated learning to improve rare-attack sample support; and (3) adversarial robustness testing against evasion attacks crafted to mimic BENIGN statistics.

VIII. Conclusion

This paper presented CyberShield, a 2-Tier Cascade NIDS that addresses class imbalance and computational efficiency in real-world network security deployment. A rigorous data-engineering pipeline eliminated three critical flaws in CI-CIDS2017, producing a pure 2,520,798-row corpus. A balanced Random Forest (Tier B) and sample-weighted XGBoost (Tier C) achieved Macro F1 scores of 0.8605 and 0.8821 respectively, vastly superior to the 0.0669 achieved by a contaminated PyTorch baseline. The hybrid deployment architecture—a native C++ ONNX Runtime engine on Android achieving sub-2 ms latency, paired with an AWS-hosted FastAPI microservice—demonstrates that production-grade NIDS can be delivered across edge and cloud from a single model corpus.

References

[1] A. K. Lakhina, M. Crovella, and C. Diot, "Diagnosing network-wide traffic anomalies," in *Proc. ACM SIGCOMM*, Portland, OR, USA, 2004, pp. 219–230.

[2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th ICISSP*, Madeira, Portugal, 2018, pp. 108–116.

[3] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE CISDA*, Nashville, TN, 2009, pp. 1–6.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[6] ONNX Community, "ONNX: Open Neural Network Exchange," GitHub, 2019. [Online]. Available: https://github.com/onnx/onnx

[7] J. Kang, Y. Yu, and Z. Zhang, "Latency-optimized on-device inference with ONNX Runtime for mobile neural networks," *IEEE Trans. Mob. Comput.*, vol. 21, no. 6, pp. 2143–2156, 2022.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD*, San Francisco, CA, 2016, pp. 785–794.

[9] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[10] S. Raschka, "Model evaluation, model selection, and algorithm selection in machine learning," *arXiv preprint arXiv:1811.12808*, 2018.